实验5.3 控制模块交互编程

学号	姓名	专业年级	课程老师
21080012078	姚嫣如	网络空间安全23	赵硕

一、实验目的

该实验为网络通信实验，根据模拟出的温度和光照场景，通过网线连接来控 制风扇和灯泡运行，了解真实设备与计算机互联的方式，培养编程能力。

通过编写数据采集及控制软件，培养利用网络通信接口获取传感数据以及对设备进行控制的编程能力，为**物联网系统综合数据采集软件**的设计与开发打下基础。

二、实验要求

根据实验内容，预先编写数据采集和控制软件，实验过程中认真调试、思考和记录，掌握**利用软件自动获取网络接口传感设备数据**以及**对设备的控制方法**， 撰写实验报告。

根据如下要求完成实验报告:

（1）将代码及解析结果附在实验报告中;

（2）写出编写程序及调试过程中遇到的主要问题和解决方法;

（3）总结实验的体会和收获

三、实验内容

（1）解析温度文件（位于“实验5.3\温度.txt”），当解析后温度数据**大于 10℃**时，通过控制模块控制风扇开启；当温度**小于等于10℃**时，控制风扇运行 关闭。

（2）解析光照强度文件（位于“实验5.3\光照强度.txt”），当解析后光照强度**小于5000Lux**时，通过控制模块控制灯泡开启；当光照度**大于10000Lux**时， 控制灯泡运行关闭。

四、实验设备

（1）12V电源一个；

（2）ZQWL-IO-5CZRE2 控制模块一个；

（3）灯泡及黑色灯泡基座一套；

（4）风扇一个；

（5）笔记本电脑一台；

（6）DC一分三电源线一根；

（7）DC-022B 直流电源线两根；

（8）黑色电线一根。

五、通信协议

ZQWL-IO-5CZRE2 控制模块有两个控制输出口，控制命令见表

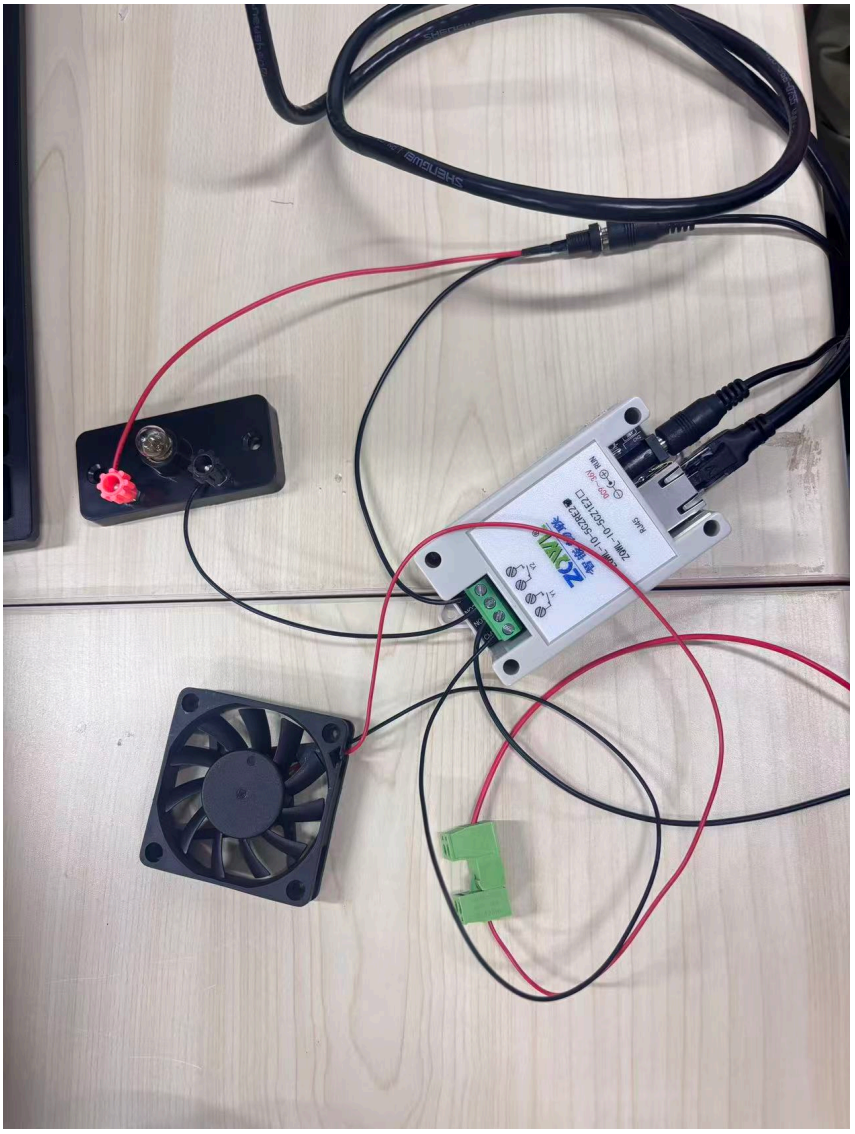
表 5.3.1 控制命令对照表

字节	1	2	3	4	5	6	7	8	9	10
COM1 启动	48	3A	01	70	01	01	00	00	45	44
COM1 关闭	48	3A	01	70	01	00	00	00	45	44
COM2 启动	48	3A	01	70	02	01	00	00	45	44
COM2 关闭	48	3A	01	70	02	00	00	00	45	44
名称	帧头		地址码	命令码	继电器序号	继电器状态	时间 TH	时间 TL	帧尾	

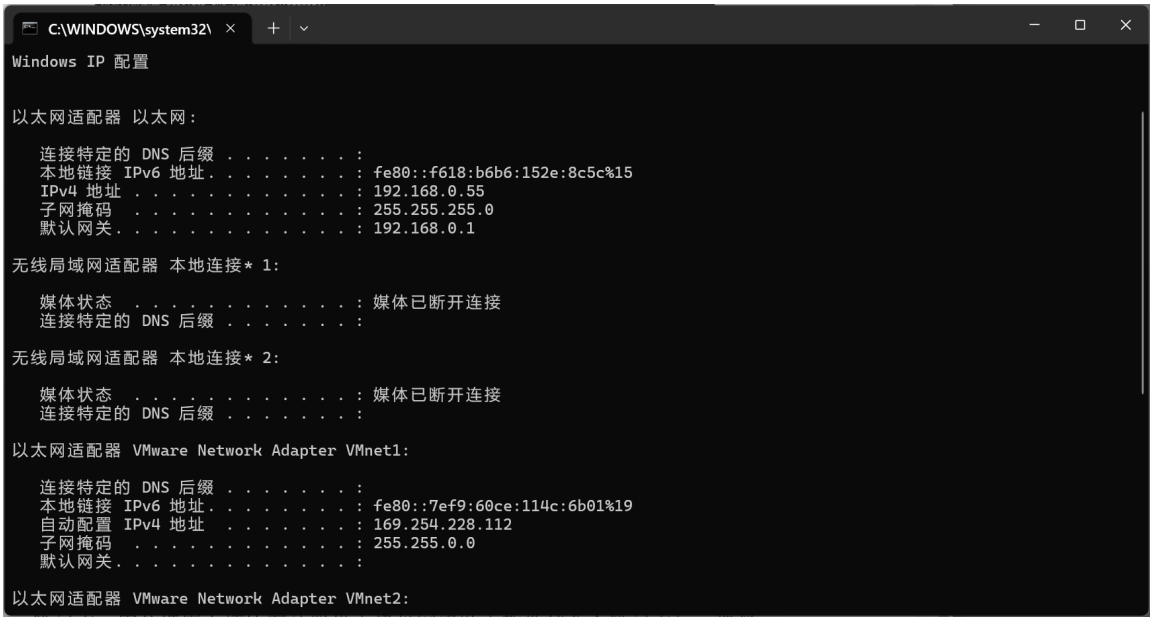
六、实验步骤

本次实验使用标有“**模块三 灯泡与风扇控制配件**”的实验模块盒：

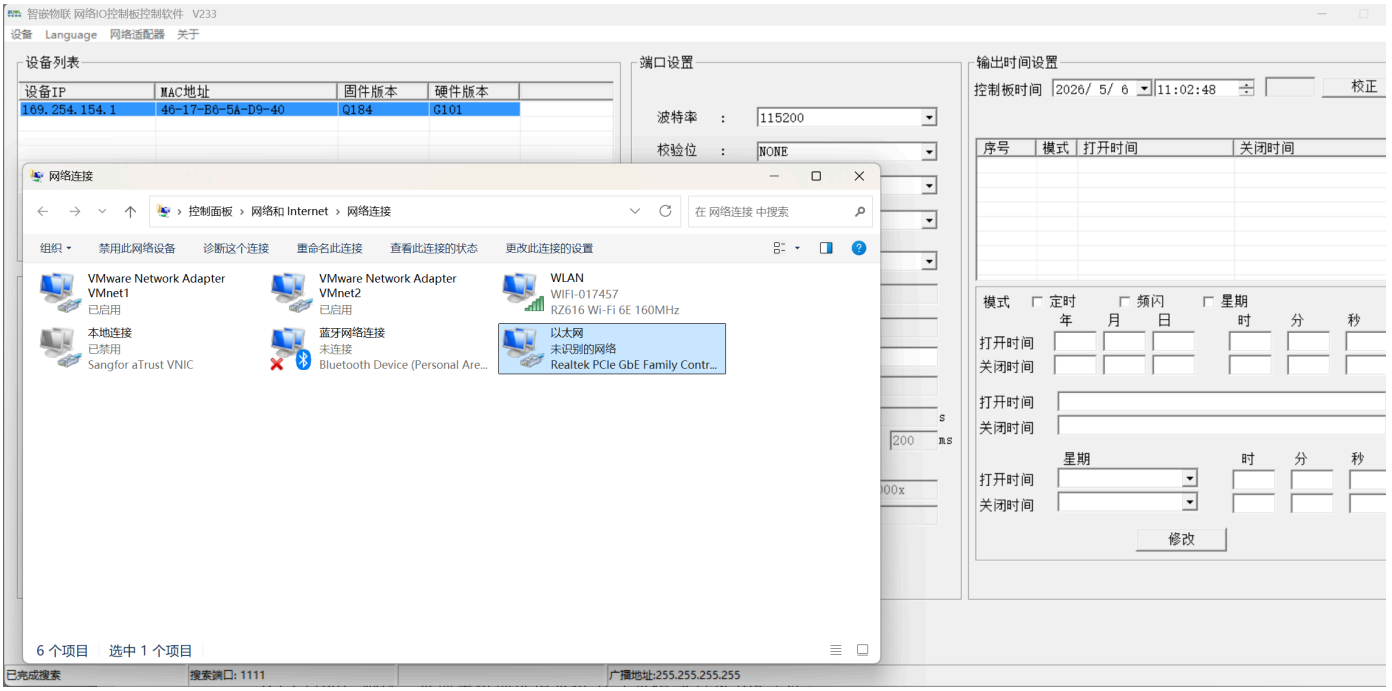
1. 电源连接：打开实验模块盒后，先拿出其中的12V电源适配器、一根DC一分三电 源线、两根DC-022B直流电源线和ZQWL-IO-5CZRE2控制模块进行电源线连接。
2. 传感器连接：将其中一个DC-022B直流电源线的黑色线路与ZQWL-IO-5CZRE2 控制模块COM1口左边端子相连，红色线路通过绿色接线端子与风扇的红色线路相连。随后，将灯泡安装在灯泡基座上，并把另一个DC-022B直流电源线的黑色线路与ZQWL-IO-5CZRE2 控制模块COM2口左边端子相连，红色线路与灯泡底座相 连，随后使用盒中黑色电线将灯泡底座与ZQWL-IO-5CZRE2控制模块COM2口右边 端子相连，形成完整线路。最后，使用网线连接控制模块与笔记本。



3. 适配器配置：打开CMD命令窗口，输入ipconfig，查看本机IP地址。

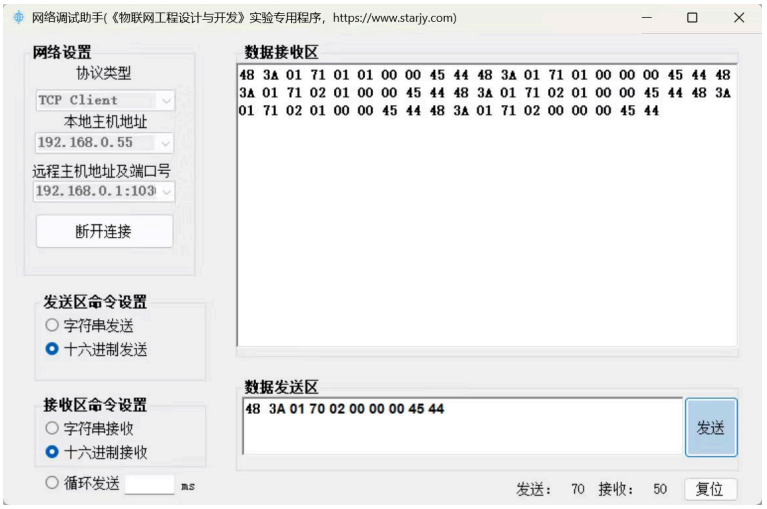


打开网络继电器控制板软件，运行该软件。先在网络适配器中选择以太网相应的接口，接着点击搜索设备

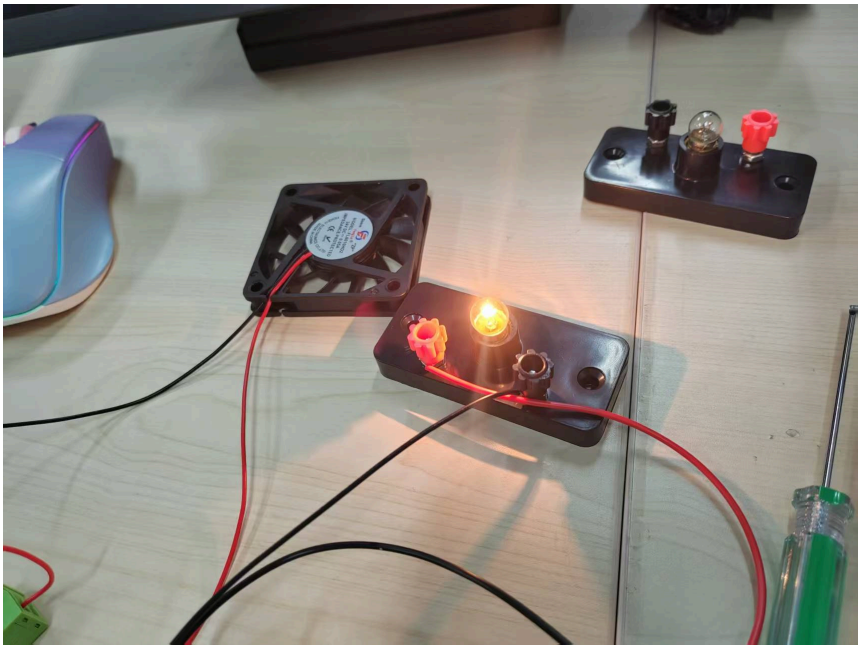


根据本机IP修改， 前三字节保持一致，最后一字节修改为1，同时将端口号处填写为1030。

4. 网络助手：打开网络助手，选择 TCP Client 协议，本地主机地址填本机IP，远程 主机地址填写刚设置的设备IP地址和端口号， 发送区命令设置选16进制发送，接收区命令设置选择字符串接收或者十六进制 接收，随后点击连接按钮。



在数据发送区编辑命令发送，尝试控制风扇和灯泡开关，在数据接收区 会显示已发送的命令。



5. 程序编写：根据实验内容编写程序，解析温度数据文件及光照度数据文件。学生需在计算机上使用编程语言(如Python、C++等)编写程序，指定远程主机的IP地 址和端口号，建立TCP连接，并根据相应控制命令发送相关指令。

   首先对温度文件进行解析，**每三秒读取一行温度信息**，并**根据温度大小对风扇进行开关操作。**（当解析后温度数据大于10℃时，通过控制模块控制风扇开启，当温度小于等 于10℃时，控制风扇运行关闭。）

   其次，对光照强度文件进行解析，**每三秒读取一行光照强度信息**，并**根据光照强度大小对灯泡进行开关操作**。（当解析后光照强度小于5000Lux时，通过控制模块控制灯泡开启，当光照度大于10000Lux时， 控制灯泡运行关闭。）

```python
# 实验5.3 控制模块交互编程 - 温度/光照解析
import socket
import time

# ====================== 配置 ======================
DEVICE_IP = "192.168.0.1"
```

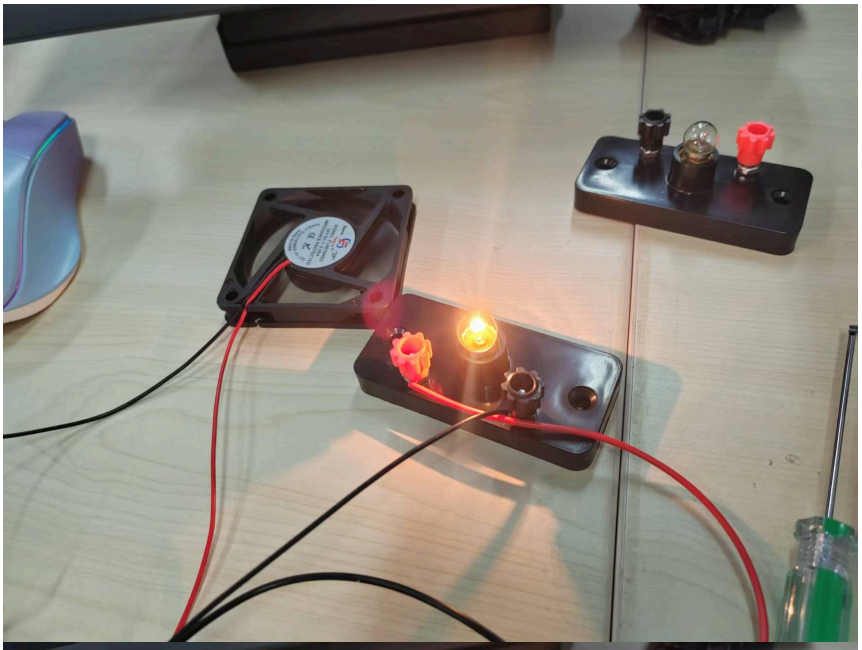
```python
DEVICE_PORT = 1030
TEMP_FILE = "温度.txt"
LIGHT_FILE = "光照强度.txt"
# ================================================

# 控制指令
FAN_ON = bytes.fromhex("483A0170010100004544")
FAN_OFF = bytes.fromhex("483A0170010000004544")
LAMP_ON = bytes.fromhex("483A0170020100004544")
LAMP_OFF = bytes.fromhex("483A0170020000004544")

# ---------------------- 温度解析（完全正确） ----------------------
# 格式：01 03 04 02 92 00 00 64 FF
# 温度值 = 第6、7字节：00 00 → 0 → 0.0℃
def parse_temp(line):
    try:
        p = line.strip().split()
        # 第6个字段：p[5]，第7个字段：p[6]
        val = int(p[5] + p[6], 16)
        return val / 10.0  # 真实温度 = 十六进制值 ÷10
    except:
        return 0.0

# ---------------------- 光照解析 ----------------------
# 格式：01 03 02 03 E8 C2 B7
# 光照 = 第4、5字节
def parse_light(line):
    try:
        p = line.strip().split()
        return int(p[3] + p[4], 16)
    except:
        return 0

# ---------------------- 按行读取（循环滚动） ----------------------
temp_lines = []
light_lines = []
temp_idx = 0
light_idx = 0

def load_lines():
    global temp_lines, light_lines
    with open(TEMP_FILE, encoding='utf-8') as f:
        temp_lines = [l.strip() for l in f if l.strip()]
    with open(LIGHT_FILE, encoding='utf-8') as f:
        light_lines = [l.strip() for l in f if l.strip()]

def get_next_temp():
```

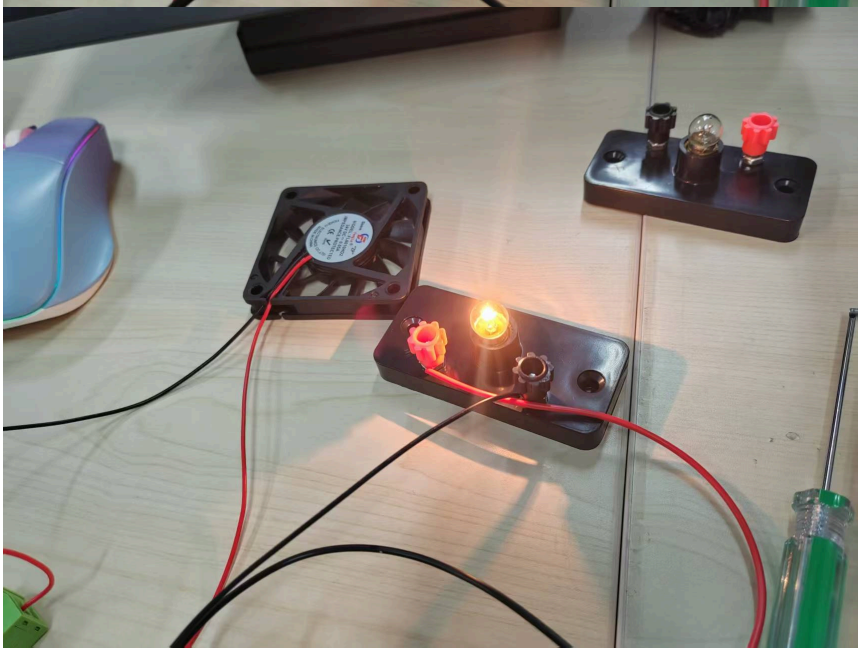
```
    global temp_idx
    line = temp_lines[temp_idx % len(temp_lines)]
    temp_idx += 1
    return parse_temp(line)

def get_next_light():
    global light_idx
    line = light_lines[light_idx % len(light_lines)]
    light_idx += 1
    return parse_light(line)

# ---------------------- TCP连接 ----------------------
def connect():
    try:
        c = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        c.settimeout(3)
        c.connect((DEVICE_IP, DEVICE_PORT))
        print("✅ TCP 连接成功")
        return c
    except Exception as e:
        print("❌ 连接失败：", e)
        return None

# ---------------------- 主逻辑 ----------------------
def main():
    load_lines()
    client = connect()
    if not client:
        return

    last_fan = None
    last_lamp = None

    print("\n=== 开始自动解析控制（逐行滚动）===\n")
    while True:
        temp = get_next_temp()
        light = get_next_light()

        # 风扇：>10℃ 开
        if temp > 10:
            if last_fan != "ON":
                client.send(FAN_ON)
                last_fan = "ON"
            fan_s = "开启"
        else:
            if last_fan != "OFF":
                client.send(FAN_OFF)
```

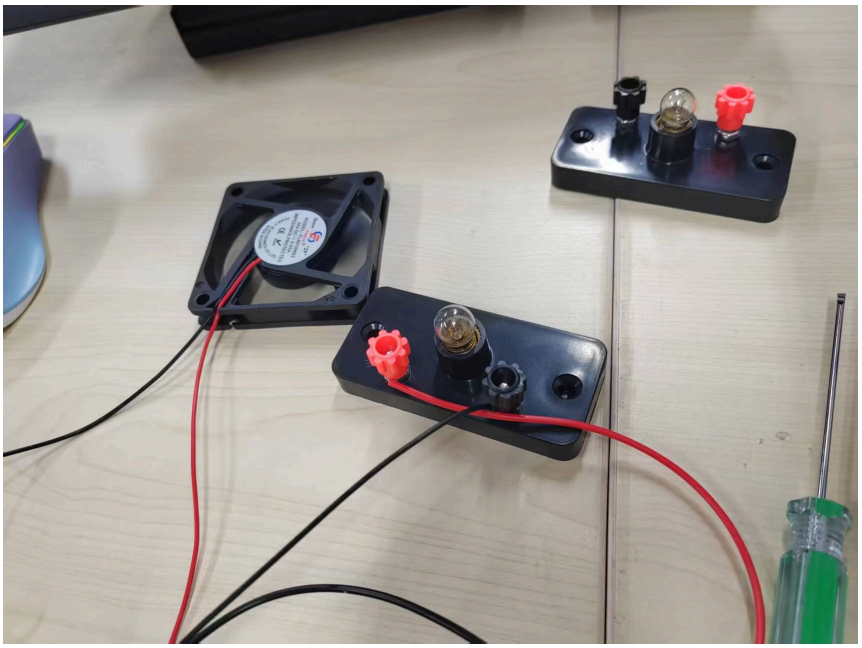
```
                last_fan = "OFF"
            fan_s = "关闭"

        # 灯：<5000开，>10000关
        if light < 5000:
            if last_lamp != "ON":
                client.send(LAMP_ON)
                last_lamp = "ON"
            lamp_s = "开启"
        elif light > 10000:
            if last_lamp != "OFF":
                client.send(LAMP_OFF)
                last_lamp = "OFF"
            lamp_s = "关闭"
        else:
            lamp_s = "保持"

        print(f"温度:{temp:5.1f}℃ | 风扇:{fan_s} | 光照:{light:6d}Lux | 灯:{lamp_s}")
        time.sleep(3)

if __name__ == "__main__":
    main()
```

运行结果如下：

```
(base) PS D:\A_junior\物联网\实验5> & D:/miniconda3/envs/myenv/python.exe d:/A_junior/物联网/实验5/实验5.3.py
✅ TCP 连接成功

=== 开始自动解析控制（逐行滚动）===

温度:  0.0℃ | 风扇:关闭 | 光照:     0Lux | 灯:开启
温度:  0.5℃ | 风扇:关闭 | 光照:  1000Lux | 灯:开启
温度: 26.6℃ | 风扇:开启 | 光照:  2000Lux | 灯:开启
温度:  1.5℃ | 风扇:关闭 | 光照:  5000Lux | 灯:保持
温度:  2.0℃ | 风扇:关闭 | 光照: 10000Lux | 灯:保持
温度:  0.1℃ | 风扇:关闭 | 光照: 15268Lux | 灯:关闭
温度:  0.6℃ | 风扇:关闭 | 光照: 20000Lux | 灯:关闭
温度: 52.3℃ | 风扇:开启 | 光照:  5944Lux | 灯:保持
温度:  1.6℃ | 风扇:关闭 | 光照:  8884Lux | 灯:保持
温度:  0.8℃ | 风扇:关闭 | 光照: 17784Lux | 灯:关闭
温度:  0.0℃ | 风扇:关闭 | 光照:     0Lux | 灯:开启
温度:  0.5℃ | 风扇:关闭 | 光照:  1000Lux | 灯:开启
温度: 26.6℃ | 风扇:开启 | 光照:  2000Lux | 灯:开启
温度:  1.5℃ | 风扇:关闭 | 光照:  5000Lux | 灯:保持
温度:  2.0℃ | 风扇:关闭 | 光照: 10000Lux | 灯:保持
温度:  0.1℃ | 风扇:关闭 | 光照: 15268Lux | 灯:关闭
温度:  0.6℃ | 风扇:关闭 | 光照: 20000Lux | 灯:关闭
温度: 52.3℃ | 风扇:开启 | 光照:  5944Lux | 灯:保持
温度:  1.6℃ | 风扇:关闭 | 光照:  8884Lux | 灯:保持
温度:  0.8℃ | 风扇:关闭 | 光照: 17784Lux | 灯:关闭
温度:  0.0℃ | 风扇:关闭 | 光照:     0Lux | 灯:开启
温度:  0.5℃ | 风扇:关闭 | 光照:  1000Lux | 灯:开启
```







七、实验心得

本次温度 / 光照解析与设备控制的编程实验，让我深入体会到了物联网设备交互中数据解析与逻辑控制的核心要点。

在数据解析环节，精准定位十六进制数据的对应字节位并完成进制转换、数值校准（如温度值÷10），让我认识到硬件数据通信格式的严谨性，一个字节的偏差就会导致解析结果完全错误，也锻炼了我对数据格式的分析和处理能力。

TCP 连接与指令发送的实现，让我掌握了基础的网络编程在设备控制中的应用，理解了 socket 通信的基本流程，以及异常处理（如连接超时、文件读取异常）对程序稳定性的重要性。

而自动控制逻辑的编写（风扇按温度、灯光按光照阈值启停），则让我体会到“状态记忆”（记录 last_fan/last_lamp）的必要性，避免了重复发送指令，也理解了工业场景中“按需控制”的核心思路。